

CURSO TÉCNICO SUPERIOR PROFISSIONAL DE PROGRAMAÇÃO DE SISTEMAS DE INFORMAÇÃO
DESENVOLVIMENTO DE APLICAÇÕES

INTRODUÇÃO À PROGRAMAÇÃO ORIENTADA A OBJETOS

CONCEITOS GERAIS

- A evolução tecnológica forçou o aparecimento do paradigma da Programação Orientada a Objetos (POO)
- Por um lado, havia a necessidade de diminuir a complexidade das aplicações, por outro, a produtividade dos programadores teria de aumentar exponencialmente
- Assim, a POO veio ajudar a simplificar projetos, quer pelas suas técnicas e conceitos de programação, quer pela reutilização de código
- Antes da POO, um programa era visto como um conjunto de instruções para desempenhar uma tarefa específica
- Com a POO, um programa representa um conjunto de instruções e objetos que interagem entre si, onde cada elemento preserva a sua individualidade



OBJETOS

- Um objeto é algo que existe no mundo real
- Um objeto é uma entidade que exhibe um determinado comportamento
- OBJETO = ATRIBUTOS + MÉTODOS
 - Os **atributos** são as características do objeto, ou seja, são as variáveis do objeto
 - Os **métodos** são as operações do objeto, ou seja, são as funções que o objeto pode realizar



CLASSES (1/3)

- As classes são a estrutura dos objetos, ou seja, representam as especificações dos objetos
- As classes são vistas como o molde para a criação de objetos
- *class* é a palavra reservada usada para criar uma classe
- Os atributos e métodos são definidos dentro da sua classe

Classe carro

Atributos (características):

- Cor
- Pneu
- ...

Métodos (funções):

- Acelerar
- Travar
- ...



CLASSES (2/3)

- Sintaxe de CLASSES

- `<acesso> class <nome_da_classe>`
- `{`
- `<bloco_de_instrucoes>;`
- `}`

```
// CLASSE
class Carro
{
    //atributos e métodos
}
```

- Sintaxe de ATRIBUTOS

- `<acesso> <tipo_dados> <nome_do_atributo>;`

```
class Carro
{
    // ATRIBUTOS
    string cor, marca, modelo;
    int cilindrada, velocidade;
}
```

CLASSES (3/3)

- Sintaxe de MÉTODOS
- `<acesso> [static] <tipo_dados_retorno> <nome_metodo> ( [<lista_parametros>] )`
 - {
 - `<declaracoes>;`
 - `...;`
 - `<bloco_instrucoes>;`
 - `...;`
 - `return <expressao>;`
 - }

```
class Carro
{
    string cor, marca, modelo;
    int cilindrada, velocidade;
    void Parar()
    {
        velocidade = 0;
    }
    void Acelerar(int quantidade)
    {
        velocidade = velocidade + quantidade;
    }
    void Travar(int quantidade)
    {
        velocidade = velocidade - quantidade;
    }
}
```

MODIFICADOR STATIC

- Indica ao compilador que o atributo ou método é definido ao nível da classe e não da instância.
- Os métodos *static* só podem chamar outros métodos *static*, não podendo aceder a métodos de uma instância sem existir um objeto.
- Na classe *Program* que contém o método *static void Main(string[] args)* todos os métodos são *static*. Esta classe é *static* por ser o ponto de entrada em que não existe ainda nenhum objeto.
- Numa classe *static* todos os métodos são *static*. Utilizado para classes utilitárias com métodos gerais a toda a aplicação que não necessitam de uma instância de um objeto.

CONSTRUTOR (1/3)

- Permite construir o objeto, tem as instruções para inicializar o objeto (e não só)
- Tem o mesmo nome da classe
- Não tem tipo de dados de retorno
 - Esta característica distingue um construtor de um método
- É o método chamado quando é utilizado o operador “*new*”
- Uma classe pode ter mais do que um construtor

CONSTRUTOR (2/3)

EXEMPLO

- Na classe Carro estão declarados dois métodos construtores de objeto
- Um sem passagem de valores, que inicializa o objeto sem valores definidos
- O segundo com passagem de valores, permite definir valores imediatamente na criação do objeto

```
class Carro
{
    string cor, marca, modelo;
    int cilindrada, velocidade;

    public Carro()
    {
    }

    //Construtor
    public Carro(string corOrigem, int cilindradaOrigem)
    {
        //Código associado à construção do objeto:
        this.cor = corOrigem;
        this.cilindrada = cilindradaOrigem;
        this.velocidade = 0;
    }

    ...
}
```

CONSTRUTOR (3/3)

EXEMPLO

```
class Carro
{
    string cor, marca, modelo;
    int cilindrada, velocidade;

    //Construtor
    public Carro(string corOrigem, int cilindradaOrigem)
    {
        //Código associado à construção do objeto:
        this.cor = corOrigem;
        this.cilindrada = cilindradaOrigem;
        this.velocidade = 0;
    }

    void Parar()
    {
        velocidade = 0;
    }
    void Acelerar(int quantidade)
    {
        velocidade = velocidade + quantidade;
    }
    void Travar(int quantidade)
    {
        velocidade = velocidade - quantidade;
    }
}
```

- Inicia alguns atributos com valores
- A keyword “this” representa a instância atual da classe, ou seja, o objeto que foi criada e que está a executar o código.

INSTANCIAÇÃO DE OBJETOS (1/2)

- Depois de definida a classe é possível criar uma instância e é possível manipular esse objeto em qualquer ponto de execução do programa
- Para criar uma instância ou objeto da classe é utilizado o operador *new*
- Sintaxe:
 - `<nome_classe> <nome_instancia> = new <nome_classe> (<argumentos>)`
- Análise sintaxe:
 - `<nome_classe>` é o nome da classe que se pretende instanciar
 - `<nome_instancia>` é o nome da nova variável-objeto que se vai criar
 - `<argumentos>` parâmetros com os quais se pode inicializar o objeto

INSTANCIAÇÃO DE OBJETOS (2/2)

```
class Program
{
    static void Main(string[] args)
    {
        Carro a = new Carro(); //Nova instância
    }
}

class Carro
{
    string cor, marca, modelo;
    int cilindrada, velocidade;

    void Parar()
    {
        velocidade = 0;
    }
    void Acelerar(int quantidade)
    {
        velocidade = velocidade + quantidade;
    }
    void Travar(int quantidade)
    {
        velocidade = velocidade - quantidade;
    }
}
```

Uma nova classe deve ser criada num novo ficheiro.

Por exemplo:
Solution Explorer->Botão do lado direito -> ADD -> CLASS
(carro.cs)



ACESSO (1/4)

CLASSES

- O modificador de acesso define o nível de proteção para atributos, métodos e classes, representando o tipo de acesso a outras classes/métodos

Nível de proteção CLASSES	Descrição
<i>Internal</i> (Só no meu Assembly)	• A classe só pode ser acedida a partir do <i>assembly</i> onde se encontra definida. • Nível de proteção definido, por omissão.
<i>Public</i> (Em todos os Assemblies)	• A classe pode ser acedida a partir de qualquer <i>assembly</i>.

Assembly representa um ficheiro ou arquivo de código compilado. Existem 2 tipos de assemblies: biblioteca (.dll) ou executável (.exe). (Cada um com um namespace diferente)



ACESSO (2/4)

ATRIBUTOS, MÉTODOS

Nível de proteção ATRIBUTOS e MÉTODOS	Descrição
Private (Apenas a minha classe)	- Só pode ser acedido a partir da classe onde foi declarado. - Nível de proteção definido, por omissão, no caso dos membros de uma classe (atributos, métodos, construtor, etc.)
Public (Todas as classes e Assemblies)	Pode ser acedido a partir de qualquer classe (inclusive fora do <i>assembly</i> onde foi definido).
Protected (Só a minha classe e família – herança, em todos os Assemblies)	Só pode ser acedido a partir da classe onde foi declarado e a partir de classes derivadas (inclusive fora do <i>assembly</i> onde foi definido).
Internal (Só no meu Assembly)	Pode ser acedido a partir de qualquer classe dentro do <i>assembly</i> (excetuando fora do <i>assembly</i> onde foi definido).
protected internal (Classes no mesmo Assembly OU Família mesmo noutro Assembly)	Só pode ser acedido a partir da classe onde foi declarado e a partir de classes derivadas (excetuando fora do <i>assembly</i> onde foi definido).
private protected (Apenas Família no mesmo Assembly)	O acesso limitado à classe que o contém ou a classes derivadas dentro do mesmo <i>Assembly</i> (disponível a partir da versão de C# 7.2).

ACESSO (3/4)

```
class Program
{
    static void Main(string[] args)
    {
        Carro a = new Carro("Branco", 1600); //Nova instância
        a.velocidade = 20; //Alteração de estado
    }
}
```

Como a variável **velocidade** é acedida a partir de uma classe diferente devemos torná-la pública.

ACESSO (4/4)

```
class Carro
{
    string cor, marca, modelo;
    public int cilindrada, velocidade;

    //Construtor
    public Carro(string corOrigem, int cilindradaOrigem)
    {
        //Código associado à construção do objeto:
        cor = corOrigem;
        cilindrada = cilindradaOrigem;
        velocidade = 0;
        Parar(); //Por ser privado só pode ser chamado dentro da classe
        Console.WriteLine("O objeto foi construído");
    }

    private void Parar()
    {
        // código
    }
}

class Program
{
    static void Main(string[] args)
    {
        Carro carro = new Carro("Branco", 1600);
        carro.velocidade = 20; //Alteração de estado
    }
}
```


MENSAGENS (1/2)

- Uma mensagem é a ação de efetuar uma chamada a um método.
- *Por exemplo, se quero que um carro pare, devo invocar o método Parar.*
- Sintaxe
 - *objeto.metodo([argumentos]);*

```
class Start
{
    static void Main(string[] args)
    {
        Carro carro = new Carro("Branco", 1600);
        carro.velocidade = 20;
        carro.Parar();
        carro = null;
    }
}
```

Pela mesma razão apontada em relação ao atributo *velocidade*, o método *Parar()* deve ter acesso *public*.

MENSAGENS (2/2)

```
class Carro
{
    string cor, marca, modelo;
    public int cilindrada, velocidade;

    //Construtor
    public Carro(string corOrigem, int cilindradaOrigem)
    {
        //Código associado à construção do objeto:
        cor = corOrigem;
        cilindrada = cilindradaOrigem;
        velocidade = 0;
    }
}
```

```
public void Parar()
{
    velocidade = 0;
}
void Acelerar(int quantidade) //método c/ 1 argumento
{
    velocidade = velocidade + quantidade;
}
void Travar(int quantidade)
{
    velocidade = velocidade - quantidade;
}
```

```
}
```

Métodos que podem ser chamados para executar ações no objeto, ou seja, que permitem o envio de **Mensagens** ao objeto.

POO - ENCAPSULAMENTO

- O encapsulamento é uma forma de "esconder" as propriedades ou métodos (mensagens) de forma a que a interface da nossa classe apenas mostre o que faz sentido.
- Outra maneira de pensar sobre encapsulamento é que ele é um escudo protetor que impede que os dados sejam acedidos pelo código fora desse escudo.
- Neste contexto surgem os métodos seletores e modificadores:
 - *Set*
 - *Get*



MÉTODO SELETOR (*GET*) E MODIFICADOR (*SET*)

- *get* (selecionar) e *set* (modificar)

```
class Carro
{
    string cor, marca, modelo;
    int cilindrada, velocidade;
    int posX;
    int posY;

    public int getPosX()
    {
        return posX;
    }
    public void setPosX(int posX)
    {
        this.posX = posX;
    }
    ...
}
```

this é uma palavra reservada.

○ *this* é uma referência ao próprio objeto.

A instrução *this.posX = posX* atribui o parâmetro contido no *posX* ao atributo *posX* do objeto.

Bom hábito usar o *this*, permite saber sempre que se trata de um atributo da classe.

PROPRIEDADES

- Em C#, existe o conceito de propriedade
- Propriedades são métodos públicos que permitem alterar o valor dos atributos do objeto
- Esses métodos são os métodos assessores (*get* e *set*) mas são apresentados como se se tratassem de um atributo
- No exemplo seguinte, o atributo cor é alterado para uma propriedade

```
class Carro
{
    string nome;
    int posX;
    int posY;
    public int Cor { get; set; }
}
```

Definir automaticamente uma propriedade no VS:

Escrever *prop* e carregar no TAB 2x. Surge de imediato a sintaxe:

```
public int MyProperty { get; set; }
```



PROPRIEDADES

- Propriedades permitem controlar o que é devolvido e o que é gravado num atributo

```
public class Carro
{
    string nome;

    0 referências
    public Carro() { }

    0 referências
    public string Nome
    {
        get
        {
            if (nome == null)
                return "";
            else
                return nome;
        }
        set
        {
            if (value != "")
                nome = value;
        }
    }
}
```

HERANÇA (1/2)

- É um dos principais conceitos de POO - a sua utilização permite a reutilização de código
 - Poupa tempo de trabalho aos programadores.
- Esta técnica de programação permite criar classes a partir de outras que já existem
- Funciona como uma hierarquia:
 - **Nível superior: classe-mãe** (é a classe original);
 - **Nível inferior: classe-filha** (é a classe gerada a partir da original).
- A classe-filha consegue herdar todos os atributos e métodos da classe original, logo não há duplicidade de código. Também é possível definir novos atributos e métodos.
- Sintaxe

```
class classe-filha : classe-mae
{
    //Aqui devem estar as instruções que definem a classe-filha
}
```

HERANÇA (2/2)

EXEMPLO

```
class CarroNovo : Carro
{
    public bool descapotavel; //Novo atributo
    public void Arrancar()    //Novo método
    {
        velocidade = velocidade + 1;
    }

    public CarroNovo(bool descapotavelOrigem) //Construtor
    {
        descapotavel = descapotavelOrigem;
    }
}
```


POLIMORFISMO

- Poli (várias) + Morfos (formas). Um objeto pode assumir diferentes formas
- O polimorfismo está associado à herança
- O conceito de polimorfismo distingue a possibilidade de dois ou mais objetos executarem a mesma ação
- Um método com o mesmo nome (mesma ação) pode originar resultados diferentes



POLIMORFISMO COM OVERLOAD

- O mesmo método (mesmo nome) com diferentes parâmetros

```
public class Calculadora
{
    0 referências
    public int Somar(int a, int b)
    {
        return a + b;
    }

    0 referências
    public int Somar(int a, int b, int c)
    {
        return a + b + c;
    }

    0 referências
    public double Somar(double a, double b)
    {
        return a + b;
    }
}
```

POLIMORFISMO COM OVERRIDE (HERANÇA)

- Substituição da execução de métodos da classe mãe.
- Palavra-Chave **virtual** deve ser utilizada para indicar que o método pode ser reescrito pela classe filha.
- O Override não é obrigatório

```
public class Animal
{
    2 referências
    public virtual void FazerSom()
    {
        Console.WriteLine("O animal faz um som");
    }
}

0 referências
public class Cao : Animal
{
    1 referência
    public override void FazerSom()
    {
        Console.WriteLine("O cão late");
    }
}

0 referências
public class Gato : Animal
{
    1 referência
    public override void FazerSom()
    {
        Console.WriteLine("O gato mia");
    }
}
```

CLASSES ABSTRATAS

- Em POO as classes abstratas são utilizadas para atribuir funcionalidades iguais a objetos diferentes, servindo de classe base
- Uma classe abstrata não pode ser instanciada, só as classes que derivam dela
- Quando uma classe tem um método abstrato, deve ser declarada como abstrata. Mas, é possível ter uma classe abstrata que não tenha métodos abstratos
- Quando existe um método abstrato (que não possui corpo), obriga a que nas classes derivadas o mesmo tenha de ser implementado, caso contrário, serão também elas classes abstratas
- Sintaxe
 - ***abstract class*** **<nome_classe_abstrata>**
 - {
 - **<bloco de instruções>;**
 - }

CLASSES ABSTRATAS

- Método **abstract**: Não tem código (corpo) e é obrigatório o override
- Método **virtual**: Tem Código no corpo e não é obrigatório o override



- Criam uma estrutura comum para várias classes
- Evitam duplicação de Código
- Obrigam as classes filhas a implementar determinados métodos
- Facilitam o polimorfismo
- Não se aplica aos atributos

```
public abstract class Animal
{
    public string Nome;

    0 referências
    public void Dormir()
    {
        Console.WriteLine("O animal está a dormir");
    }

    1 referência
    public abstract void FazerSom();
}

0 referências
public class Gato : Animal
{
    1 referência
    public override void FazerSom()
    {
        Console.WriteLine("O gato mia");
    }
}
```

O método abstract teve de ser implementado na classe filha